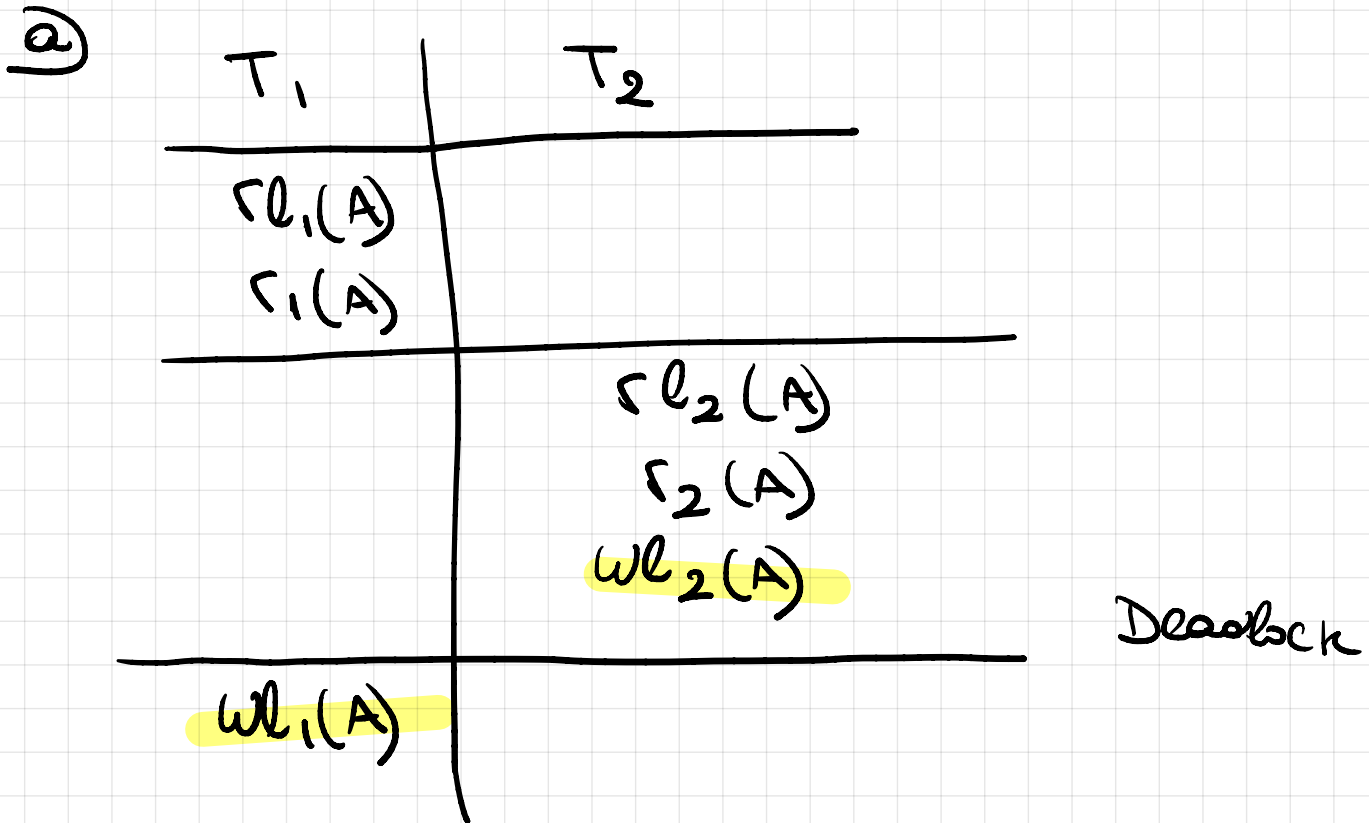


3.9. Tomando las siguientes transacciones

$T_1 = rl_1(A); r_1(A); wl_1(A); w_1(A); u_1(A)$

$T_2 = rl_2(A); r_2(A); wl_2(A); w_2(A); u_2(A)$

- (a) Proponer un entrelazamiento que de un *deadlock* como resultado.
- (b) Reescribir las transacciones, pero ahora, usando un modelo que soporte *update lock* y verificar si puede resolverse el *deadlock* anterior.



T_2 : Tiene el lock de lectura y se puede esperar. Poder obtener el lock para escribir, el cual no puede obtenerlo porque T_1 tiene el $sl_1(A)$

T_1 : Ya tiene el lock de lectura y se puede esperar el lock de escritura, pero no lo puede obtener porque T_2 tiene $rl_2(A)$

Ambos están esperando un recurso que depende del otro: Deadlock.

b)

T_1	T_2
$ul_1(A)$ $s_1(A)$	
	$ul_2(A)$ ← se puede esperar a que T_1 libere el lock
$ul_1(A)$ $w_1(A)$ $u_1(A)$	
	$s_2(A)$ $ul_2(A)$ $w_2(A)$ $u_2(A)$

ya no hay más deadlock.